

TP

Test Plan

MotorsRent

Riferimento	
Versione	1.0
Data	10/11/2025
Destinatario	Azienda MotorsRent S.r.l.
Presentato da	Ingenito Davide Sorrentino Luca Teodonno Francesco
Approvato da	

Revision History

Data	Versione	Descrizione	Autori
05/12/2025	0.1	Prima stesura	Team
07/12/2025	0.5	Definizione Test cases	Team
09/12/2025	1.0	Revisione finale	Team

Revision History.....	2
1 Introduzione.....	3
2 Relazione con altri documenti.....	3
3 Panoramica del sistema	3
4 Features da testare/da non testare	4
5 Pass/Fail criteria.....	4
6 Approccio.....	5
7 Materiale di testing.....	5
8 Test cases.....	6
9 Test schedule.....	10

1. Introduzione

Il presente documento ha l'obiettivo di descrivere ed analizzare le attività di Testing per la piattaforma MotorsRent. Il fine è quello di garantire che ogni aspetto funzioni in modo corretto rispetto ai requisiti.

Sono state pianificate attività di testing per le seguenti gestioni:

- **Gestione Utente** (Registrazione, Login, Profilo)
- **Gestione Catalogo Auto** (Ricerca, Filtraggio, Scheda Dettaglio)
- **Gestione Commerciale** (Richieste Preventivo, Leasing)
- **Gestione Amministrativa** (CRUD Veicoli)

2. Relazione con altri documenti

Per la corretta individuazione dei test case, si fa riferimento agli altri documenti prodotti.

- **Relazioni con il Requirements Analysis Document (RAD):**
I test case sono elaborati in relazione ai requisiti funzionali (RF_FU, RF_FA, RF_FV) e non funzionali (RNF) presentati nel RAD v0.6.
- **Relazioni con il System Design Document (SDD):**
I test case rispettano la suddivisione in sottosistemi (MVC) presentata nell'SDD v1.0.
- **Relazioni con il Object Design Document (ODD):**
I test di unità e integrazione seguiranno le specifiche delle classi e dei DAO descritti nell'ODD v1.0.

3. Panoramica del Sistema

Il sistema proposto basa la sua architettura sul pattern MVC (Model-View-Controller) in un'architettura Three-Tier.

- **Front-end (View):** HTML5, CSS3, JSP per la generazione delle interfacce.
- **Back-end (Controller/Model):** Applicazione Java basata su Servlet container.
- **Database: DBMS Relazionale (MySQL)** gestito tramite driver JDBC/SQL.

4. Feature da testare/da non testare

Features da testare

Gestione Utenza (RF_FU_2)

- Registrazione nuovi utenti (Ospite -> Cliente)
- Login/Logout per i 3 ruoli (Cliente, Venditore, Amministratore)
- Gestione Area Personale

Gestione Catalogo (RF_FU_1, RF_FA_1)

- Visualizzazione elenco auto
- Filtraggio (Marca, Prezzo, Anno)
- Visualizzazione scheda dettaglio

Gestione Commerciale (RF_FU_3, RF_FU_4, RF_FV_1, RF_FV_2)

- Invio Richiesta Preventivo
- Invio Richiesta Leasing (Simulazione)
- Gestione richieste (lato Venditore)

Gestione Amministrativa (RF_FA_1)

- Inserimento/Modifica/Rimozione Veicoli

Features da non testare

Le funzionalità di cui non si andrà ad effettuare le attività di testing riguardano requisiti esclusi dal perimetro contrattuale (SOW):

- Pagamento online
- Integrazione con sistemi di terze parti (ACI, finanziarie)
- Gestione officina e manutenzione

5. Pass/Fail criteria

L'esito di un test case è valutato mediante un oracolo, basandosi sui requisiti. Un test ha successo (**pass**) se l'output ottenuto corrisponde all'output atteso. Fallisce (**fail**) in caso contrario.

Tutto il testing sarà considerato valido se tutti i seguenti vincoli saranno rispettati:

- Testare il 100% dei requisiti funzionali critici.
- Caricamento catalogo in meno di 3 secondi (RNF_P_1).
- Assenza di crash di sistema (Eccezioni non gestite).

6. Approccio

Il testing dell'intero sistema si compone di tre fasi: testing di sistema, testing di integrazione e testing di unità.

Testing di Sistema

Verrà utilizzato un approccio black-box per validare i flussi utente (Scenari RAD).

- Functional Testing: Validazione dei requisiti funzionali (es. Login, Ricerca).
- Performance Testing: Verifica dei tempi di risposta del catalogo.

Testing di Integrazione

Verrà utilizzato un approccio bottom-up, partendo dal layer Model (DAO) fino al layer Controller. Verrà utilizzato JUnit e Mockito per isolare le dipendenze (es. mock del database per testare i Controller).

Testing di Unità

La strategia prevede di testare ogni metodo delle classi del sistema (esclusi getter/setter banali). Verrà utilizzato JUnit 5.

7. Materiale di Testing

L'hardware necessario è un computer di sviluppo standard configurato come server locale (Apache Tomcat + MySQL) e client web (Browser Chrome/Firefox).

8. Test cases

L'approccio per la definizione dei test frame sarà il **Category Partition**.

8.1 Gestione amministrativa

8.1.1 Inserimento auto nel catalogo

Parametro: Prezzo	
Nome Categoria	Scelte per la categoria
Validità [PR]	1. Prezzo ≤ 0 = false [error] 2. Prezzo > 0 = true [PROPERTY PR_OK]

Parametro: Anno Immatricolazione	
Nome Categoria	Scelte per la categoria
Validità Temporale [AN]	1. Anno $>$ AnnoCorrente = false [error] 2. Anno $\leq$ AnnoCorrente = true [PROPERTY AN_OK] 3. Anno < 1900 = false [error]
Parametro: Marca	
Nome Categoria	Scelte per la categoria
Lunghezza [MA]	1. Lunghezza < 2 = false [error] 2. Lunghezza ≥ 2 AND ≤ 50 = true [PROPERTY MA_OK]
Parametro: Chilometraggio	

Nome Categoria	Scelte per la categoria
Validità [KM]	1. Km < 0 = false [error] 2. Km >= 0 = true [PROPERTY KM_OK]

Test Case ID	Test frame	Esito
TC_ADM_1.1	PR1	Errato: Prezzo negativo o nullo
TC_ADM_1.2	PR2, AN1	Errato: Auto del futuro
TC_ADM_1.3	PR2, AN3	Errato: Anno troppo vecchio
TC_ADM_1.4	PR2, AN2, MA1	Errato: Marca troppo corta
TC_ADM_1.5	PR2, AN2, MA2, KM1	Errato: Chilometri negativi
TC_ADM_1.6	PR2, AN2, MA2, KM2	Corretto: Auto inserita

8.2 Gestione commerciale

8.2.1 Richiesta Leasing

Parametro: Durata Mesi	
Nome Categoria	Scelte per la categoria
Intervallo [DM]	1. Mesi < 12 = false [error] 2. Mesi >= 12 AND <= 60 = true [PROPERTY DM_OK] 3. Mesi > 60 = false [error]
Parametro: Anticipo	
Nome Categoria	Scelte per la categoria
Importo [ANT]	1. Anticipo < 0 = false [error] 2. Anticipo > PrezzoAuto = false [error] 3. Anticipo >= 0 AND < PrezzoAuto = true [PROPERTY ANT_OK]

Test Case ID	Test frame	Esito
TC_LEA_2.1	DM1	Errato: Durata troppo breve
TC_LEA_2.2	DM3	Errato: Durata troppo lunga

TC_LEA_2.3	DM2, ANT1	Errato: Anticipo negativo
TC_LEA_2.4	DM2, ANT2	Errato: Anticipo superiore al valore auto
TC_LEA_2.5	DM2, ANT3	Corretto: Richiesta inviata

8.3 Autenticazione

8.3.1 Login Utente

Parametro: Email	
Nome Categoria	Scelte per la categoria
Formato [EM]	1. Non rispetta il formato dell'email = false [error] 2. Rispetta il formato dell'email = true [PROPERTY EM_OK]
Parametro: Password	
Nome Categoria	Scelte per la categoria
Corrispondenza [PW]	1. Password errata = false [error] 2. Password corretta = true [PROPERTY PW_OK]

Parametro: Ruolo	
Nome Categoria	Scelte per la categoria
Validità Ruolo [RL]	1. Ruolo non riconosciuto o nullo = false [error] 2. Ruolo valido (Cliente, Venditore, Amministratore) = true [PROPERTY RL_OK]

Test Case ID	Test frame	Esito
TC_LOG_3.1	EM1	Errato: Formato email invalido
TC_LOG_3.2	EM2, PW1	Errato: Credenziali non valide
TC_LOG_3.3	EM2, PW2, RL1	Errato: Accesso negato (Ruolo utente non identificato o corrotto).
TC_LOG_3.4	EM2, PW2, RL2	Corretto: Accesso consentito con reindirizzamento alla Dashboard corretta.

7. Testing Schedule

Le attività di pianificazione del testing sono avvenute subito dopo la fase di design. La scrittura dei casi di test avverrà in contemporanea con lo sviluppo del codice. L'esecuzione dei test avverrà sia durante che dopo l'implementazione del sistema.